


# ESP32 顯示屏：活動前導讀

| | |
|---|---|
| ↗ 🍺 Upcoming Maker 聚會一覽 | 🍺 22 Aug 2026 (六) 社群聚會：Hobbyist 興趣發燒友 |
| ↗ 🚩 實體活動一覽 | 💪 Maker 同好「一齊整」：ESP32 顯示屏 |
| Last edited time | September 7, 2026 2:22 PM |
| Created | September 3, 2026 8:03 AM |
| ↗ 🏷️ 標籤 Tag | STEAM教育, Hobbyist, 興趣發燒友, DIY, 動手造教育, AI |
| 分類 | DIY 資訊 |

💡 **歡迎來到 Maker 同好「一齊整」！**
這份導讀旨在幫助大家在動手之前，建立簡單的硬體與軟體概念！

## 目錄


**一、 運作原理：這個顯示屏是如何運作的？**

二、前置知識 (硬件) - 螢幕是如何顯示資訊的？

三、前置知識 (硬件) - ESP32 是什麼?

四、前置知識 (軟件) - ESP32 裡面的程式碼在忙什麼?

**五、 一般的製作步驟 (活動不會依這個次序製作)**

六、 現場需要共同解決的 5 大問題

# 一、 運作原理：這個顯示屏是如何運作的？

ESP32顯示屏本質上是一個 **IoT（物聯網）裝置**，運作流程如下：

```
[線上開放數據 / API 伺服器] ──(Wi-Fi 網際網路)──> [ESP32 微控制器] ──(實體排線)──> [TFT LCD 螢幕]
```

1. **連線**：ESP32 開機後，會自動連接你家中的 Wi-Fi（透過程式預設的帳密或網頁設定）。
2. **拿取數據**：ESP32 定時向網絡上的公開資料庫（例如九巴、香港天文台、港鐵等）發送 HTTP Request，把最新的實時數據 (JSON 格式) 拿取。
3. **繪製**：ESP32 將數據解析，並結合預先存放在ESP32的內部記憶體的背景圖片與字型，將圖像與文字繪製到螢幕上。

▼ 💡 **專有名詞小字典**

- **IoT (Internet of Things, 物聯網)**：指讓燈泡、螢幕、感應器等日常生活設備連接網路並傳遞數據的技術。
- **ESP32**：一種價格便宜、帶有 Wi-Fi 和藍牙功能的高性能微型控制晶片。
- **API (Application Programming Interface)**：機構（如九巴、天文台）開放給外界程式讀取數據的「數位窗口」。
- **HTTP Request**：晶片向網路伺服器發送的「資料索取請求」。
- **JSON**：一種輕量且結構清晰的文字格式，常用於網路傳輸資料（例如：`{"bus_time": "5 mins"}`）。

# 二、前置知識 (硬件) - 螢幕是如何顯示資訊的？

在開始動手之前，我們需要先建立一個核心概念：**對微型晶片（ESP32）來說，螢幕上呈現的所有東西，本質上都是「點陣圖」。**

1. **螢幕本質是「幾萬個小燈泡」**：
   2.8 吋螢幕上有數萬個小像素點（RGB 燈泡）。晶片本身無法直接「憑空印字」，它必須精確告知每個燈泡「在什麼位置、顯示什麼顏色」。
2. **畫面上主要由兩種元素組合而成：**
   a. **圖（背景與圖示）—— 需要「解壓後的原始資料」**：
   我們平時用的 JPG 圖片經過高度壓縮，微型晶片算力有限、解不開。因此我們使用未經壓縮的**點陣圖檔（`.bmp`）**，晶片才能像讀取畫布一樣，直接把顏色刷上螢幕。
   b. **字（動態數據）—— 需要「字型字典」**
   晶片天生只認識數字與代碼，並不認識「中文」。要顯示中文字，晶片必須從預先存好的字型檔（`.vlw`）中，查找這個字對應的「文字小圖案」，再像蓋印章一樣把圖案拼貼到螢幕上。

3. **晶片的記憶體空間：**
   上面提到的「字型檔」與「背景圖片」檔案較大，而螢幕本身並沒有儲存檔案的能力。因此，我們必須預先將素材儲存在 ESP32 內部的獨立記憶體空間（Flash/SPIFFS） 中，運作時再由 ESP32 即時讀取並發送給螢幕顯示。

## 三、前置知識 (硬件) - ESP32 是什麼?

ESP32 是一塊 MCU (微控制器)，能獨立執行我們編寫的程式碼，並透過邊緣的金屬針腳 **GPIO（通用輸入輸出腳位）** 發送電訊號，用來控制外接設備（如螢幕、馬達、繼電器）或讀取感測器數據（如溫濕度、光敏)。晶片內部帶有 4MB 的 **Flash Memory（閃存記憶體)，**這是晶片的資料儲存空間，特性是即是切斷電源，存放在裡面的資料也不會消失。

ESP32最大的特色是自帶「接收」與「發射」功能的微型網絡設備，能夠在以下兩種網路模式靈活切換或組合。

- **功能一：身為 Station 模式 (STA模式)**
  ESP32 可以像你的手機或平板一樣，自動搜尋並連接現有的 Wi-Fi 路由器。連上網路後，它就能透過 **HTTP / HTTPS 請求** 發送指令給網絡上的伺服器，自動取得各類遠端資料。或是將本地感測器（如溫濕度）收集到的數據上傳至雲端；甚至能向網路上的其他智能設備發送控制指令。
- **功能二：身為 SoftAP 模式 (熱點模式)**
  當 ESP32 找不到預設的 Wi-Fi ，或初次開機需要設定時，它能自己發射出一個 Wi-Fi 訊號（例如名為 `Smart-Display-Setup` 的熱點)。你的電腦或手機連上這個熱點後，就能開啟 **Web 設定頁面**，直接在手機瀏覽器上為 ESP32 進行設定。
- **雙模併行 (AP + STA 模式) 與 藍牙 (Bluetooth / BLE)：**
  ESP32 甚至支援同時開熱點又連 Wi-Fi；而內建的藍牙功能（BLE 低功耗藍牙)，則常用於在沒有 Wi-Fi 的環境下與手機快速配對、近距離傳輸數據或進行初期設定。

## 四、前置知識 (軟件) - ESP32 裡面的程式碼在忙什麼?

硬件是裝置，軟件便是控制裝置運作的思維邏輯。當ESP32通電後，內部程式碼會輪流執行以下6項工作：

> 💡 [開機通電] ➔ 1. 初始化系統 ➔ 2.讀取 Flash 的檔案系統 SPIFFS (SPI Flash File System) ➔ 3. 架設網頁伺服器 ➔ 4. 連 Wi-Fi / 索取數據 ➔ 5. 拆解 JSON 包裹 ➔ 6. 繪製刷新畫面 ➔ (循環回 Step 4)

1. **初始化系統 (System Initialization)**

**初始化系統（System Initialization）** 是 ESP32 通電或重置（Reset）後執行的**第一個步驟**，將晶片調整為「具備確定性、可執行高階程式碼的穩定狀態」。具體而言，晶片剛通電時會先執行 `setup()` (即「開機僅執行一次」的初始化區塊)，設定處理器頻率與記憶體空間。

由於理解晶片針腳無法自動辨認外接元件，必須透過 `pinMode()` 主動指定各金屬針腳(GPIO)是「讀取訊號（輸入）」還是「輸出控制電壓 （輸出）」。然後，執行 `Serial.begin(115200)` 初始化 UART 通訊控制器，並將傳輸速率（波特率）設定為115200bps，建立 ESP32 與開發電腦（如 Arduino IDE 監控視窗）之間傳送除錯訊息的數據管道。

▼ ***對應的 C++ 基礎語法概念***

> 💡 **前置小知識**：「函數」(Function) 源自數學概念。在 C++ 基礎語法中，「函數」(Function) 是將一系列指令封裝為可重複使用並獨立執行的單元；它可接收傳入的資料 (引數 Arguments / 參數 Parameters) 作為輸入，經過邏輯運算後輸出結果（回傳值 Return Value）。若沒有函數，每次執行任務時均需重複編寫底層指令，更無法因應輸入資料而靈活控制硬體。

透過初始化系統的指令，初學者可以建立以下 3 個 C++ 的函數概念：

- **函數與程式入口點 (Functions & Entry Point)** `setup()`：在大多數程式語言中，程序的執行始於一個特定的入口點。這個入口點是程式開始執行的地方，通常是一個函數。`setup()` 及其後用大括號 `{ }` 包起的程式碼，只會在開機時被系統依序執行一次。
- **引數與狀態設定 (Arguments & State)** `pinMode()`：C++ 中有一系列的函數（如 pinMode、digitalWrite、delay 等）都是能夠設定引數來向硬體發出具體指令，用以改變其「狀態」（State）。呼叫函數時，填入括號 `( )` 內部的具體數值或設定值，稱為「引數」（Arguments）。
- **物件與方法 (Objects & Methods)** `Serial.begin()`：在 C++ 中，有些函數不是獨立運作的，而是**專屬於某個具體設備或工具**，這種專屬函數稱為「方法」（Method），**而該設備本身則稱為**「物件」（Object）。例如 `Serial` 代表晶片內部的序列埠對講機（物件），而 `.begin()` 則是這台對講機專屬的「啟動功能」（方法）。中間使用點號 . 連接，代表**「指定 Serial 這台設備，執行 .begin() 這個動作」**，括號內的數字（引數）則用來設定它的運作速度。

- 變數類型 (`int`, `float`, `String`, `bool`)
- 控制結構 (`if-else`, `for`, `while`, `setup()`, `loop()`)
- 記憶體觀念（全域變數、 `static` 作用域、 `&` 指標與引用）
- 字串操作 (`indexOf()`, `substring()`, `sprintf()`)

2. **讀取 Flash 的檔案系統 SPIFFS (SPI Flash File System)**

在初始化 ESP32的過程中，程式碼會以 `SPIFFS.begin()` 方法 (method) 嘗試掛載檔案系統，即是將Flash Memory與檔案系統 (SPIFFS) 建立連接。因此，若掛載成功，程式便能透過預設的路徑，讀取**ESP32 的網頁控制介面，**讓使用者以手機或電腦透過 Wi-Fi 連線 ESP32作設定。

### ▼ 💡 專有名詞小字典

- **SPIFFS（閃存檔案系統）**：全稱為 *SPI Flash File System*。Flash 記憶體只是一大塊未整理的原始數據區（Raw Data）；SPIFFS 就像是晶片裡的「檔案總管」，在 Flash 中劃分出專屬空間。它的作用是讓你在寫程式時，能直接用「檔名」指令硬體去讀取特定檔案（例如在程式碼寫下：「讀取 `background.bmp` 圖片」或「打開 `font.vlw` 字型」），晶片就能精確抓出該素材並顯示在螢幕上。

### ▼ *對應的 C++ 基礎語法概念*

> 💡 **前置小知識**：在 C++ 基礎語法中，程式不僅能依序執行指令，還能根據檢查結果來判斷接下來要執行哪一段程式碼。這稱為條件判斷（主要使用 if 句式，如同生活中的「如果下雨，就帶傘；否則不帶」）。

透過第二個步驟的指令，初學者可以建立的基礎程式觀念 - if 敘述 **(If Statement)**：

- **雙向選擇**：if ... else （如果...否則...）
  根據判斷結果進行二選一：條件成立時執行第一段，不成立時則執行第二段。例子：

```
if (SPIFFS.begin(true)) {
Serial.println("SPIFFS Mounted Successfully.");
} else {
Serial.println("SPIFFS Mount Failed!");
}
```

- **單向判斷**：if (如果...就...)：當條件成立（結果為 true）時，才會執行括號 { } 內部的程式碼；若條件不成立（結果為 false），則直接跳過。例子：

```
// If mounting fails, format automatically and retry; if still failed, handle the error
if (!SPIFFS.begin(true)) {
Serial.println("SPIFFS Mount Failed!");
return; // Exit setup() to prevent executing further file logic
}
```

- 多重選擇：if ... else if ... else （多條件判斷）
  當有多個可能性需要依序檢查時使用。例子：

```
if (!SPIFFS.begin(true)) {
Serial.println("SPIFFS Mount Failed!");
} else if (!SPIFFS.exists("/index.html")) {
Serial.println("SPIFFS Mounted, but /index.html not found!");
} else {
Serial.println("SPIFFS Ready: /index.html found.");
}
```

- `if (!SPIFFS.begin())`

`SPIFFS.begin()` 會回傳一個布林值（`true` 代表成功，`false` 代表失敗）。驚嘆號 `!` 為邏輯非（NOT）運算子，用於將結果反轉；當掛載失敗回傳 `false` 時，`!false` 轉為 `true`，從而觸發 `if` 區塊 `{ }` 內部的錯誤處理程式碼。

- **物件方法與檔案操作 (Objects & File Methods) —— `SPIFFS.open(path, mode)`**

  深化「物件與方法」的理解。`SPIFFS` 是一個代表 Flash 檔案系統的物件，而 `.open()` 則是其專屬的檔案操作方法。透過傳入檔案路徑（如 `"/index.html"`）與讀取模式（如 `"r"` 代表 Read），該方法會回傳一個檔案物件以供後續資料讀取。

- **字串常數與路徑表述 (String Literals & File Paths) —— `"/index.html"`**

  理解 C++ 中用雙引號 `" "` 包裹的文字稱為「字串常數」（String Literals）。在 SPIFFS 檔案系統中，所有檔案路徑均需以斜線 `/` 開頭，代表根目錄下的檔案位置。

3.

## 五、 一般的製作步驟 (活動不會依這個次序製作)

1. **實體接線**：把線將 ESP32 與 TFT LCD 螢幕連接妥當。
2. **環境配置**：準備一部電腦， 並已安裝能夠驅動ESP32的軟體及工具包，包括： Arduino IDE、晶片驅動程式（CH340/CP2102）以及驅動程式庫（`TFT_eSPI`）。
3. **Flash 寫入**：利用擴充工具，將自訂圖片（`.bmp`）與字型檔（`.vlw`）上傳至 ESP32 內部的 Flash/SPIFFS。
4. **程式修改與燒錄**：修改 Wi-Fi 帳號密碼及目標數據 API（例如車站 ID、城市名稱、股票代碼），燒錄至晶片，最後裝入 3D 列印外殼。
5. **外殼組裝與擺放**：將晶片與螢幕固定於 3D 列印外殼中，配置 USB 供電線，完成最終擺放與安裝。

> 💡 **步驟相關名詞小字典**
>
> - **Arduino IDE**：電腦上的「寫程式與上傳工具」。我們用它來編寫 C/C++ 程式碼，並將程式發送到 ESP32 晶片中執行。
> - **CH340 / CP2102（晶片驅動程式 Driver）**：讓電腦能夠透過 USB 電線「認得並溝通」ESP32 晶片的轉接軟體。如果沒裝驅動程式，電腦只會把晶片當成純充電設備而無法傳輸程式。
> - **TFT_eSPI（函式庫 Library）**：別人預先寫好、免費分享出來的「顯示工具程式」。它把控制螢幕幾萬個燈泡的複雜底層工作預先準備好，讓晶片知道如何執行「畫線」、「顯示圖片」，以及從字型檔（`.vlw`）中找出文字圖案並「印」到螢幕上。

> - **.vlw（字型資料檔）**：專門給 TFT_eSPI 讀取的「中文字型圖庫/字典」。它裡面儲存了預先轉好的文字圖案數據；TFT_eSPI 收到顯示文字的指令時，就會去 `.vlw` 檔案裡找出對應的文字圖案並蓋到螢幕上。

## 六、 現場需要共同解決的 5 大問題

- ⚠️ **驅動程式與連接埠識別**：電腦無法偵測到 ESP32 開發板（通常是未裝 CH340/CP2102 驅動程式，或使用了純充電 USB 線）。
- ⚠️ **螢幕黑屏或白屏**：實體接線錯位、杜邦線接觸不良，或 TFT_eSPI 庫檔的腳位參數設定不正確。
- ⚠️ **SPIFFS 檔案上傳失敗**：晶片的閃存分區設定錯誤、燒錄擴充插件不相容，或是上傳時佔用了序列埠（Serial Port）。
- ⚠️ **API 數據解析失敗**：數據源參數（例如車站 Location/Stop ID）填寫錯誤、Wi-Fi 帳密有誤，或網路連線逾時（Timeout）。
- ⚠️ **實體安裝與電源問題**：杜邦線過長或外殼尺寸有微差導致塞不進 3D 列印外殼；或使用的 USB 線供電不足（ESP32 連接 Wi-Fi 時瞬間電流較大），導致晶片不斷重新開機。

**一、 C++ 程式語言基礎（語法層面）**
• **變數與資料型態**：理解 `int`（整數）、`float`（小數）、`String`（文字）、`bool`（真/假）的差別。
• **控制結構**：`if-else` 判斷式、`for` 迴圈、`while` 迴圈。
• **函數（Functions）**：什麼是輸入參數（如 `drawBmp(filename, x, y...)`）、什麼是回傳值（`return`）。
• **變數作用域（Scope）**：理解 `static` 變數（為什麼函數結束後資料不會消失）與全域變數。
• **指標與引用（Pointers & References）**：`&tft` 符號的意思（傳遞記憶體位址給工具箱）。
• **字串處理技巧**：`indexOf()`（文字搜尋）、`substring()`（文字擷取）、`sprintf()`（格式化輸出）。

**二、 硬體與單片機原理（ESP32 / Arduino 層面）**
• **Arduino 生命週期**：`setup()`（開機只執行一次）與 `loop()`（無窮無限重複執行）。
• **GPIO 與 PWM 訊號**：`pinMode()` 與 `analogWrite()` 控制背光亮度的原理（用 0-255 控制電壓/亮度）。
• **通訊協定（SPI）**：ESP32 是如何透過幾根線與 SD 卡和螢幕「高速溝通」的。
• **記憶體架構（Flash vs RAM）**：
  ◦ **Flash**：變數存進 `Preferences`（斷電資料不遺失）。
  ◦ **RAM**：`Sprite` 繪圖佔用的暫存空間（開機才有，關機消失）。
• **Serial 序列埠除錯**：`Serial.print()` 如何把訊息印到電腦螢幕上用來除錯。

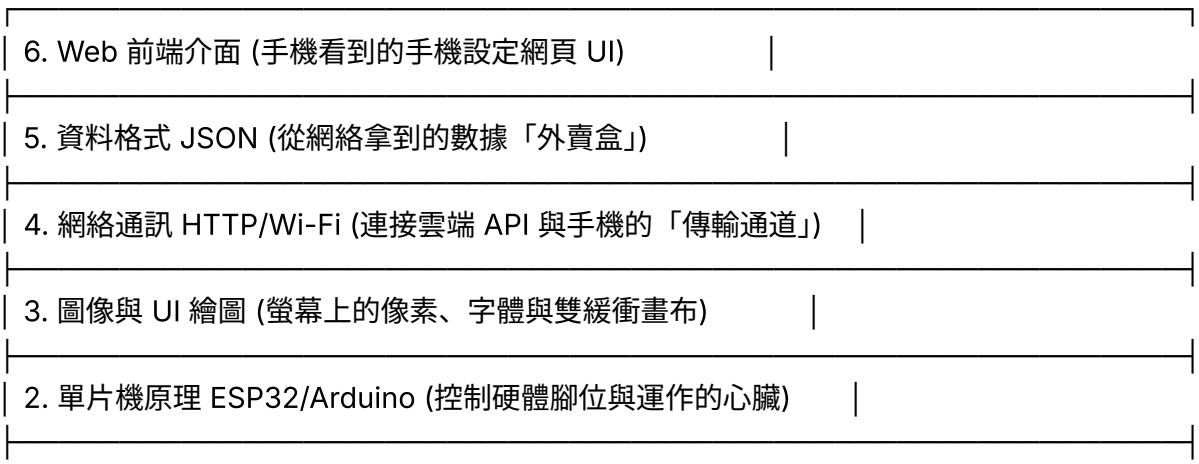
**三、 圖像與 UI 顯示原理（螢幕繪圖層面）**

• **像素與座標系**：螢幕的 $(X, Y)$ 軸方向（左上角為 $(0,0)$）。
• **顏色格式（RGB565）**：為什麼用 16-bit 數字代表顏色（紅 5 位、綠 6 位、藍 5 位）。
• **BMP 圖檔結構**：什麼是「檔頭 (Header) 54 位元組」以及「像素顛倒儲存（從下往上畫）」。
• **雙緩衝防閃爍（Sprite 概念）**：為什麼直接畫在螢幕會閃爍，而在記憶體（Sprite）畫好再一瞬間推（Push）到螢幕就不會閃。
• **向量/位圖字型（`.vlw`）**：字體如何載入、對齊方式（`MC_DATUM` 中間對齊 vs `TL_DATUM` 左上角對齊）。

**四、 網絡通訊基礎（Web & Network 層面）**
• **Wi-Fi 連線與 IP 位址**：ESP32 如何作為 Station（連家用 Wi-Fi）或 Access Point（自己發射熱點）。
• **HTTP 協定**：什麼是 `GET` 請求（向伺服器要資料）、什麼是 `POST` 請求（把資料傳給伺服器）。
• **HTTP 狀態碼**：`200` 代表成功、`404` 代表找不到頁面。
• **NTP 時間同步**：透過網路時間伺服器（`pool.ntp.org`）校準 UTC+8 香港時間。
• **mDNS 網域解析**：為什麼輸入 `[http://KMB-ETA.local](http://KMB-ETA.local)` 可以找到 ESP32。

**五、 資料格式與解析（Data Format 層面）**
• **JSON 格式**：理解字串中的 `{}`（物件）、`[]`（陣列）、`"key": "value"`（鍵值對）。
• **JSON Parsing（解析）**：如何使用 `ArduinoJson` 程式庫把一長串文字轉成程式能讀取的數字與字串。
• **時間字串拆解**：把 ISO 時間（如 `2026-08-29T12:30:00`）拆成年、月、日、時、分，並用 `mktime()` 計算剩餘分鐘數。

**六、 Web 前端基礎（手機網頁介面層面）**
• **HTML 基礎**：`<form>` 表單、`<select>` 下拉選單、`<input>` 輸入框。
• **CSS 基礎**：UI 樣式設定（顏色、圓角 `border-radius`、卡片排版）。
• **Basic JavaScript**：`fetch()` 非同步向 API 抓資料、動態更新網頁選單。



## 1. C++ 語言基礎 (貫穿全部、用來跟晶片溝通的「文字工具」)